

Ingeniería del Software de Componentes y Sistemas Distribuidos

SEGUNDA PRUEBA PRÁCTICA – PRAC 2

Presentación

En la PRAC 1 se ha diseñado una solución para el caso **Photo&Film4You** basada en microservicios, usando modelos de representación y patrones de diseño. En esta PRAC 2 se pide implementar un subconjunto de este diseño a partir de programar 3 microservicios considerando diferentes aspectos, como, por ejemplo: estado propio, comunicación síncrona, lógica de negocio, consistencia de los datos y despliegue en local.

Para elaborar esta práctica se formulan 2 ejercicios a partir del caso **Photo&Film4You**:

1. Implementar tres de los microservicios diseñados en la PRAC1, según un conjunto de consideraciones técnicas y la documentación de las evidencias.
2. Razonar cómo resolver el acceso a los microservicios desde diferentes clientes.

La PRAC 2 abarca los contenidos del libro "Microservices Patterns" (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En esta PRAC 2 se trabaja las siguientes competencias del Grado en Ingeniería Informática:

- Saber construir aplicaciones informáticas mediante técnicas de desarrollo, integración y reutilización.
- Saber proponer y evaluar diferentes alternativas tecnológicas para resolver un problema concreto.

Objetivos

Los objetivos de la PRAC 2 son:

- Implementar microservicios centrados en la lógica de negocio y el acceso a datos.
- Desplegar microservicios con estado propio e intercomunicación asíncrona.
- Aplicar conceptos básicos de la arquitectura de microservicios.
- Dominar las tecnologías básicas necesarias para la implementación de microservicios.

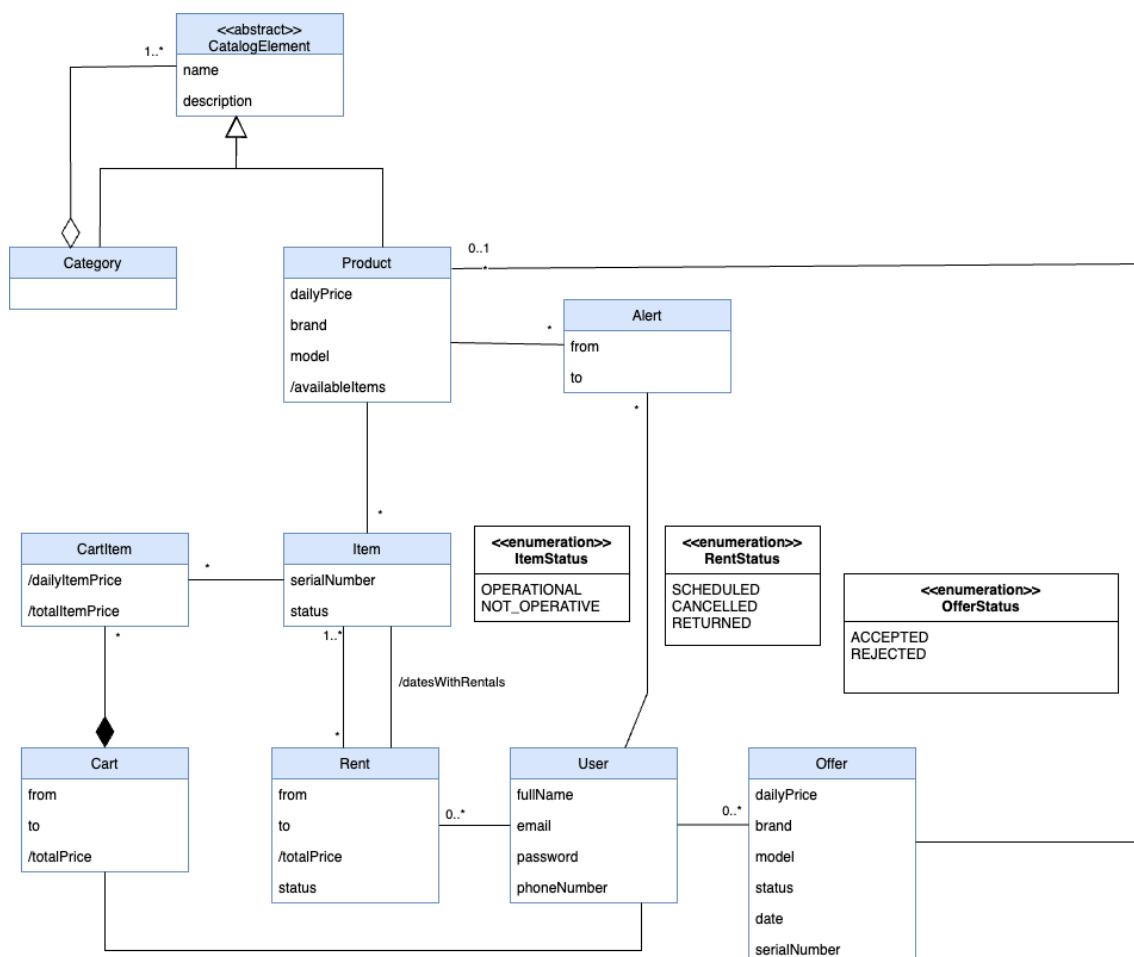
Descripción de la PRAC 2

Ejercicio 1 (8 puntos)

El Ejercicio 1 consta de dos partes, la primera parte consistirá en la implementación de un software, y la segunda parte en la documentación de las evidencias de lo que se ha implementado. A continuación, se describen las tareas concretas a realizar en cada una de las dos partes.

Primera parte (6 puntos)

Consiste en implementar un subconjunto de los microservicios que se han diseñado en la PRAC1. Debe basarse en la solución publicada por la PRAC anterior y por tanto se parte del siguiente diagrama del modelo general del dominio, del que se pueden extraer las principales entidades (con sus atributos) y relaciones:



El subconjunto de servicios a implementar se compone de los servicios *Product Catalog*, *User* i *Notification*:

- El microservicio *Product Catalog* servirá tanto para dar cobertura a las diversas operaciones que los administradores de la plataforma necesitan para gestionar el inventario (crear/eliminar productos, secciones/subsecciones y unidades de producto) como para las consultas del catálogo de productos que realizarán los usuarios.
- El microservicio *User* sirve para gestionar las altas, los datos personales de los usuarios, y la configuración de alertas. Los usuarios deben poder dar de alta alertas para un producto concreto y un rango de fechas. Cuando una unidad de ese producto quede libre para este rango de fechas, debe emitirse un mensaje a la cola para que otro servicio se encargue de hacer efectiva la alerta (microservicio *Notification*).
- El microservicio *Notification* tiene la finalidad de emitir notificaciones a los usuarios interesados en un producto concreto, cuando se añaden unidades o alguna queda libre para ser alquilada. Este microservicio debe escuchar la cola de mensajes de forma que cuando se libere una unidad de producto para una fecha en la que algún usuario tenga configurada una alerta, a este usuario se le mande un correo electrónico con el aviso.

De acuerdo con lo descrito, estos tres microservicios tendrán como mínimo las siguientes operaciones:

Microservicio *Product Catalog* (catálogo de productos):

- crear sección/subsección
- crear producto
- crear unidad de producto
- eliminar producto
- cambiar el estado de una unidad de producto a operativo/no-operativo
- consulta de secciones/subsecciones por nombre
- consulta de secciones/subsecciones por descripción
- consulta de secciones/subsecciones por sección “padre”
- consulta de productos por nombre
- consulta de productos por sección
- consulta del detalle de un producto
- consulta del detalle de una unidad
- crear una oferta
- actualizar el estado de una oferta
- consulta de ofertas que ha realizado un usuario

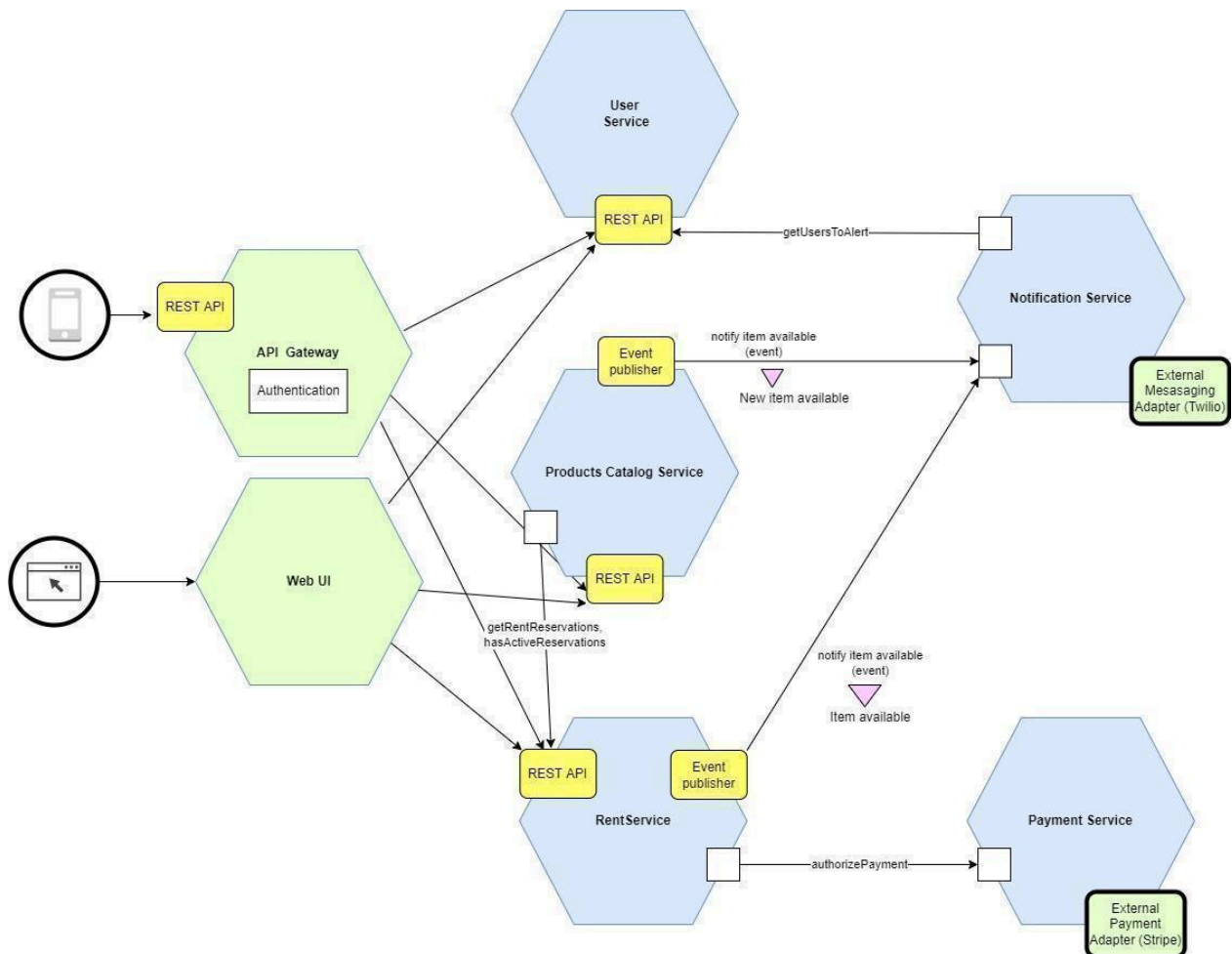
Microservicio *User* (gestión de usuarios):

- alta de usuario
- alta de alerta
- consulta de las alertas de un usuario para un intervalo concreto de fechas
- consulta de las alertas por producto y fecha
- consulta de usuarios a alertar para un producto y fecha (operación *getUsersToAlert*)
- consulta del detalle de una alerta

Microservicio *Notification* (notificaciones automáticas a usuarios):

- enviar notificación (operación de sistema, no disponible para los usuarios)

En el siguiente diagrama se puede ver cómo debe ser la colaboración entre los diversos servicios del sistema:



Consideraciones previas

Para poder conseguir la flexibilidad, resiliencia y rendimiento que caracteriza esta arquitectura, es necesario cumplir una serie de requerimientos:

- Cada microservicio:
 - tiene que poder ejecutarse independientemente,
 - tiene que tener estado propio (y acceder de manera independiente a la BBDD),
 - implementa sólo su lógica de negocio,
 - expone su contrato (definición de la API),
 - gestiona su propio *log*.

- Siempre que sea posible, la comunicación entre los diversos microservicios debe ser asíncrona y por medio de colas de mensajes.
- Para ciertas operaciones, puede ser necesaria la colaboración entre varios microservicios para obtener información. En este caso, los microservicios pueden realizar peticiones a otros microservicios por medio de llamadas REST.
- Cosas que quedan fuera del alcance del proyecto en la fase piloto que se pide implementar:
 - Autenticación: no será necesario ningún tipo de autenticación para ninguna de las operaciones, se da por hecho que esta parte de la funcionalidad del sistema la proporciona el API Gateway (que no debe implementarse).
 - Service Discovery & Registration: no se debe implementar ningún tipo de funcionalidad de registro y búsqueda de servicios, se da por hecho que sólo existe una única instancia, en una ubicación conocida.
 - Envío de correos: para no tener que implementar esta parte, de poco valor didáctico dentro de la PRAC, os sugerimos que simuleis su comportamiento con el uso de Mocks. Se debe poder demostrar al menos una línea de log cada vez que se envíe una notificación (por ejemplo, mostrar mediante el log del microservicio *Notification* una línea de log con nivel INFO que diga “Se ha notificado por correo el usuario XXX YYY ZZZ que existe una unidad disponible del producto AAA”).

Plataforma de desarrollo

- Entorno de desarrollo: IntelliJ IDEA / Eclipse
- JDK: OpenJDK 11
- Framework: Spring / Spring Boot
- Gestión de dependencias y build: Maven
- Servidor HTTP: Tomcat (integrado por defecto con Spring Boot)
- Librería llamadas REST: Jersey
- Comunicación entre servicios: Apache Kafka
- BBDD: PostgreSQL
- Implementación ORM: Hibernate
- Generación de la documentación del API OpenAPI v3: springdoc-openapi
- Log: Log4j2
- Mocks: Mockneat (para generar datos pseudo-aleatorios)

Punto de partida y recursos

Para tener un punto de partida común y homogéneo, y poder centrar los esfuerzos en el desarrollo de los microservicios, junto con este enunciado se proporciona un enlace a un repositorio público (<https://github.com/UOC-ISCD-SPRING-2024>) con los siguientes recursos:

- Un archivo *docker-compose.yml* para instalar la infraestructura básica para la implementación de este ejercicio y un archivo *README.md* con instrucciones para arrancarla.
- Esqueletos de aplicación Spring Boot con las funcionalidades básicas para generar:
 - un archivo java jar ejecutable con todas las dependencias contenidas en el proyecto (para poder hacer debug localmente);
 - un archivo de imagen docker con el microservicio y todas sus dependencias (para poder hacer la entrega).
- Un conjunto de enlaces a documentación, guías y tutoriales relativos tanto a las herramientas a utilizar como a la propia implementación de los microservicios solicitados (se colgarán en el aula de Laboratorio).

Cada uno de los esqueletos de aplicación que se proporcionan son un proyecto Spring Boot completo pre-configurado con las dependencias mínimas necesarias para hacer toda la implementación que se pide en este enunciado. Esto no significa que no se puedan añadir otras dependencias si os parece adecuado, pero con lo que hay basta para hacer toda la implementación.

Con los contenedores del archivo *docker-compose* arrancados, todos los esqueletos de microservicio tendrían que poder ponerse en marcha y conectar a la infraestructura tal como se entregan con esta PRAC2. Además, aprovechando que los esqueletos llevan su propio servidor HTTP empotrado, se ha incluido un módulo *springdoc-openapi* que auto-genera y publica el archivo de definición del microservicio (API) en formato *JSON/OpenAPI v3*, así como una pequeña interfaz web *SwaggerUI* que permite hacer llamadas a las operaciones de los microservicios.

Las direcciones para acceder a estas funcionalidades son:

- archivos de definición de los servicios:
 - *Product Catalog*: <http://localhost:18081/v3/api-docs/>
 - *User*: <http://localhost:18082/v3/api-docs/>
- interfaces web SwaggerUI para pruebas:
 - *Product Catalog*: <http://localhost:18081/swagger-ui/index.html>
 - *User*: <http://localhost:18082/swagger-ui/index.html>

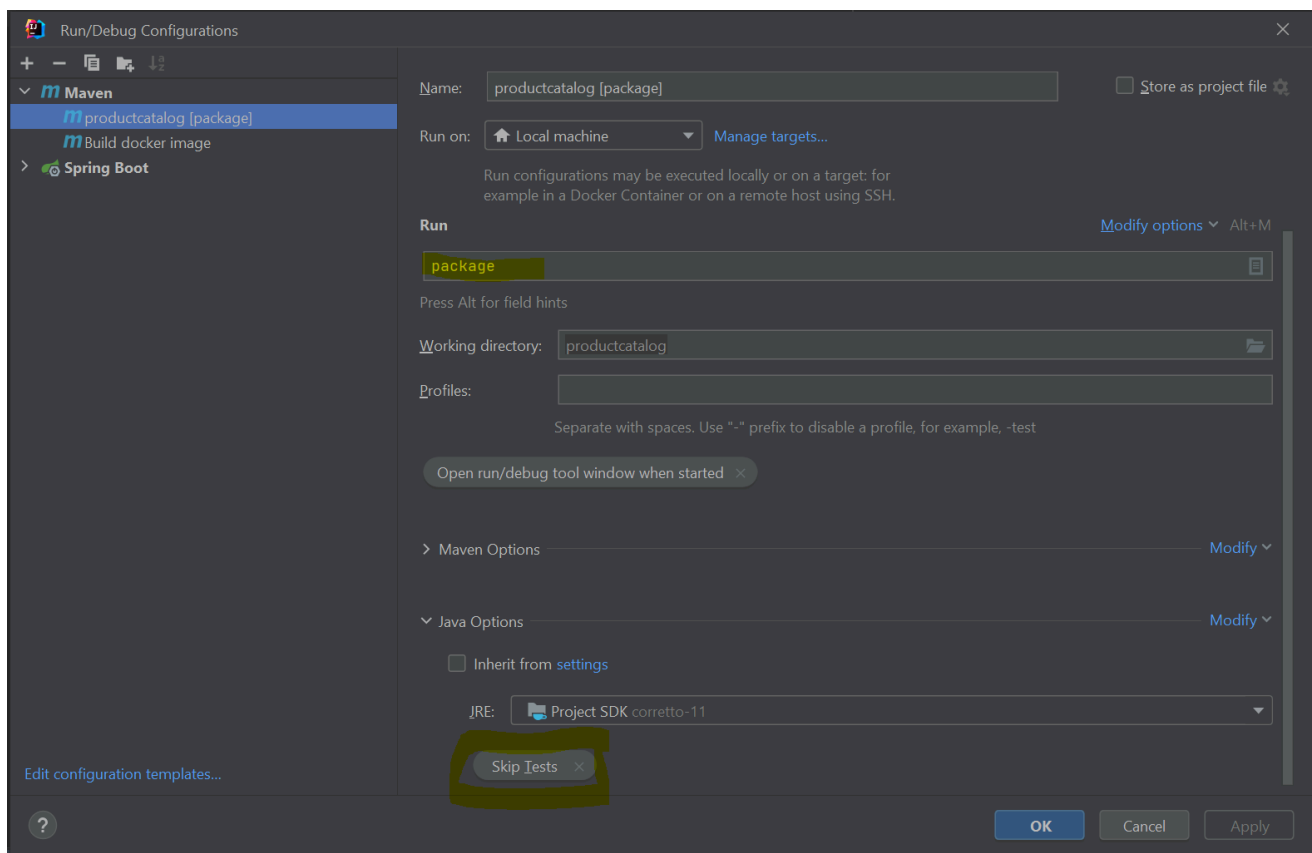
Cómo se puede ver, estos ejemplos forman una base bastante completa desde donde empezar el desarrollo y, por lo tanto, se tendría que evitar hacer modificaciones en sus componentes básicos (excepto pequeñas configuraciones, etc.), y solo añadir las funcionalidades que se piden en este enunciado. Encontraréis más información y apoyo técnico en el Laboratorio de la asignatura.

Entregable

Especificaciones técnicas de la aplicación a entregar

El entregable de esta parte del ejercicio será un único archivo ZIP que tiene que contener:

- Todo el código y archivos de configuración del proyecto en un **archivo ZIP** (dentro del ZIP “principal”).
- Un **archivo JAR** ejecutable (generado por el propio esqueleto del proyecto entregado) para cada uno de los servicios, que contendrá todas las clases y librerías necesarias (dependencias) para ejecutar el servicio en cuestión. Este archivo se puede generar a partir del esqueleto de aplicación proporcionado con una ejecución maven y el paquete “goal”.



- Un archivo **readme.txt** con la siguiente información:
 - instrucciones para poner en marcha la infraestructura necesaria para la plataforma (en caso de que sea diferente a la que se proporciona con docker compose);
 - cualquier configuración necesaria para poder ejecutar los servicios;
 - el comando concreto para ejecutar cada servicio por separado (si no se han realizado cambios, debería bastar con "java -jar productcatalog-VERSION.jar", etc.)

Solución

Se puede acceder a una implementación de referencia de los microservicios especificados desde los repositorios públicos GitHub localizados en las direcciones:

- <https://github.com/UOC-ISCD-SPRING-2024/spring-2024-productcatalog-solution>
- <https://github.com/UOC-ISCD-SPRING-2024/spring-2024-user-solution>
- <https://github.com/UOC-ISCD-SPRING-2024/spring-2024-notification-solution>

Segunda parte (2 puntos)

Para poder demostrar que se han logrado los objetivos de la implementación, tendréis que proporcionar evidencias del correcto funcionamiento de los microservicios. Para elaborar estas evidencias os proponemos utilizar la aplicación Postman, una herramienta que os permitirá importar el archivo de definición del servicio (accesible desde la ruta *v3/api-docs* de los tres microservicios) y generar una colección que contendrá todas las operaciones disponibles. Como mínimo se realizará un ejemplo de cada tipo de llamada con datos “reales” una vez puestos en marcha los microservicios.

Entregable

El entregable de esta parte del ejercicio será el propio archivo de la colección con los ejemplos de llamada para cada una de las operaciones implementadas en la primera parte.

OpenAPI definition

users

POST create User

GET get All Users

GET get User By Id

GET get Users To Alert

alerts

GET

{{baseUrl}}/users/...

Params Headers (1) Body

Query Params

Key

Key

Body Headers (5)

Pretty Raw Preview JSON

```

1 [
2   {
3     "id": 1,
4     "fullName": "Juan Palomo",
5     "email": "juanpa@gmail.com",
6     "password": "12345",
7     "phoneNumber": "654233521"
8   },
9   {
10    "id": 2,
11    "fullName": "Francisco Pérez",
12    "email": "fperez@gmail.com",
13    "password": "blabla",
14    "phoneNumber": "612446666"
15  },
16 ]

```

OpenAPI definition

users

POST create User

GET get All Users

GET get User By Id

GET get Users To Alert

alerts

POST

{{baseUrl}}/users

Params Authorization Headers (10) Body Pre-request Script Tests Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

```

1 {
2   "fullName": "Isabel Martinez",
3   "email": "imartinez@gmail.com",
4   "password": "isa2002",
5   "phoneNumber": "912325536"
6 }

```

Body Cookies Headers (6) Test Results

Pretty Raw Preview Visualize JSON

```

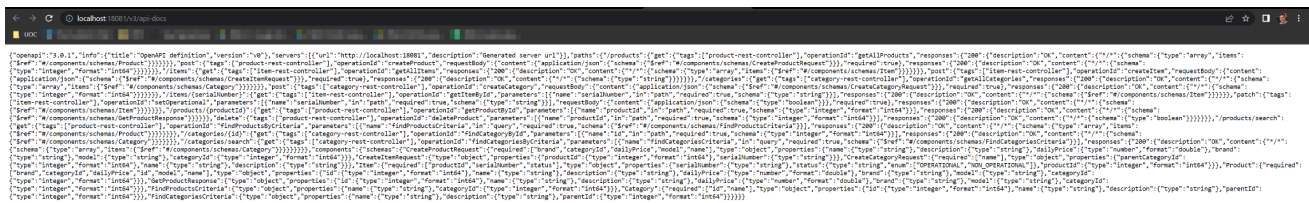
1 6

```

Solución

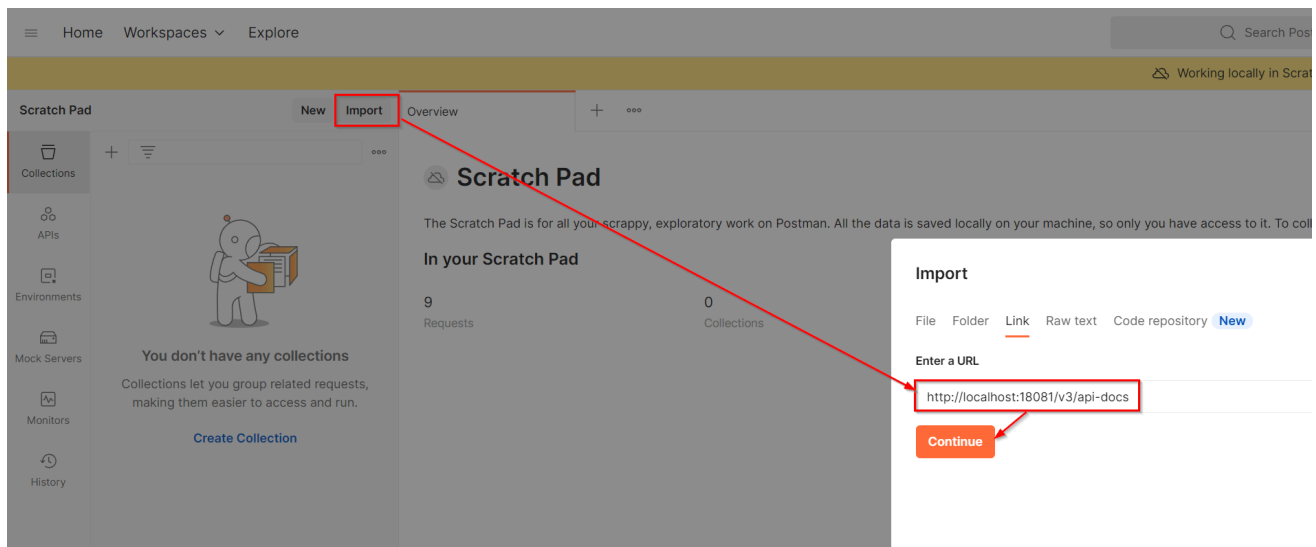
Una vez hecha la implementación, se puede obtener el “contrato” o la definición de cada uno de los servicios en formato JSON en la dirección `v3/api-docs`.

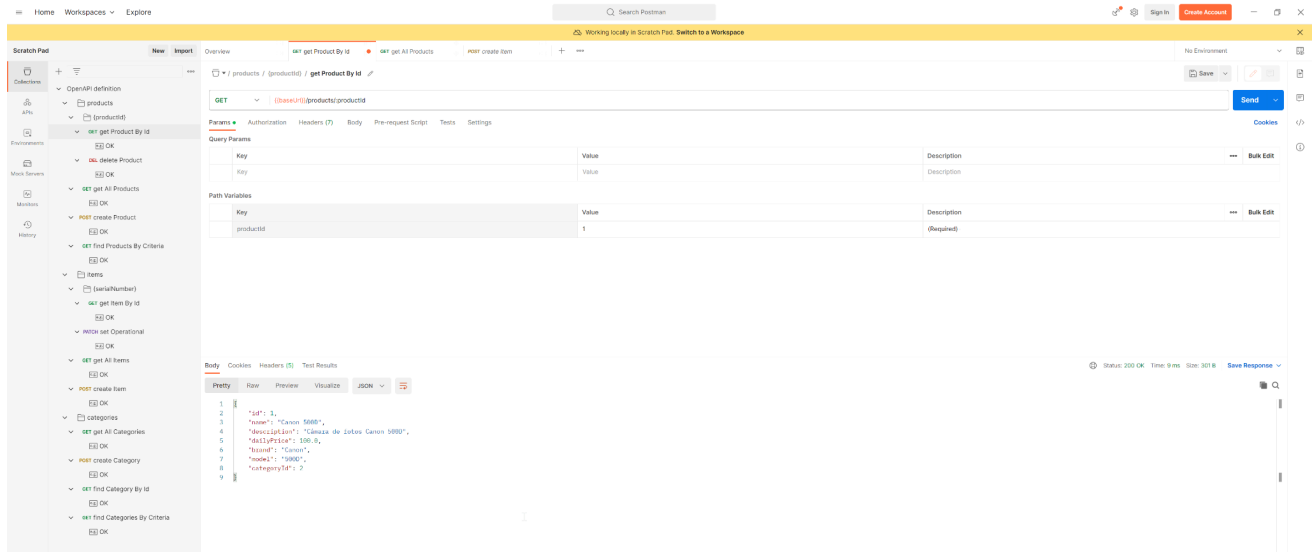
- <http://localhost:18081/v3/api-docs> (productCatalog)
- <http://localhost:18082/v3/api-docs> (user)



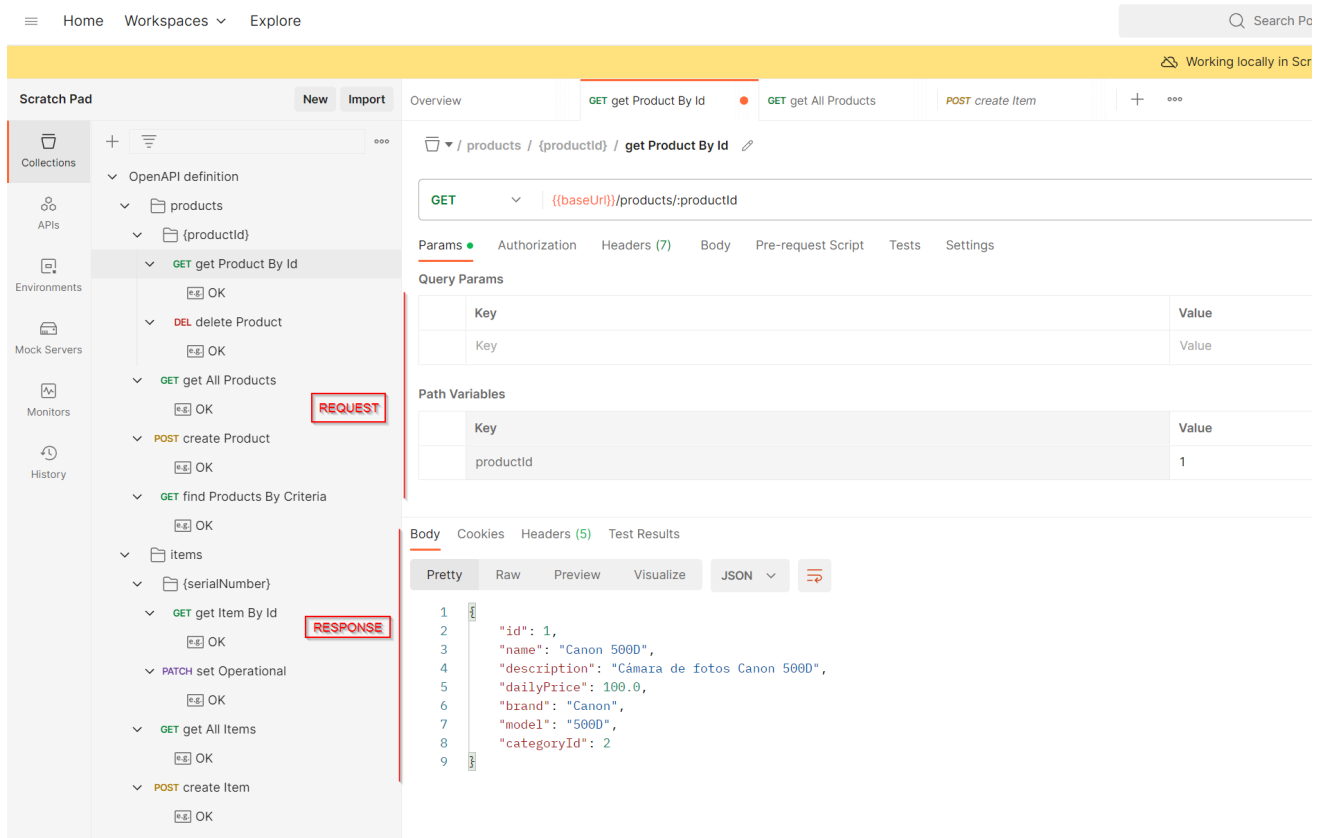
El microservicio notification no expone ninguna operación y por lo tanto, aunque se puede acceder a su interfaz swaggerUI, ofrece un contrato vacío.

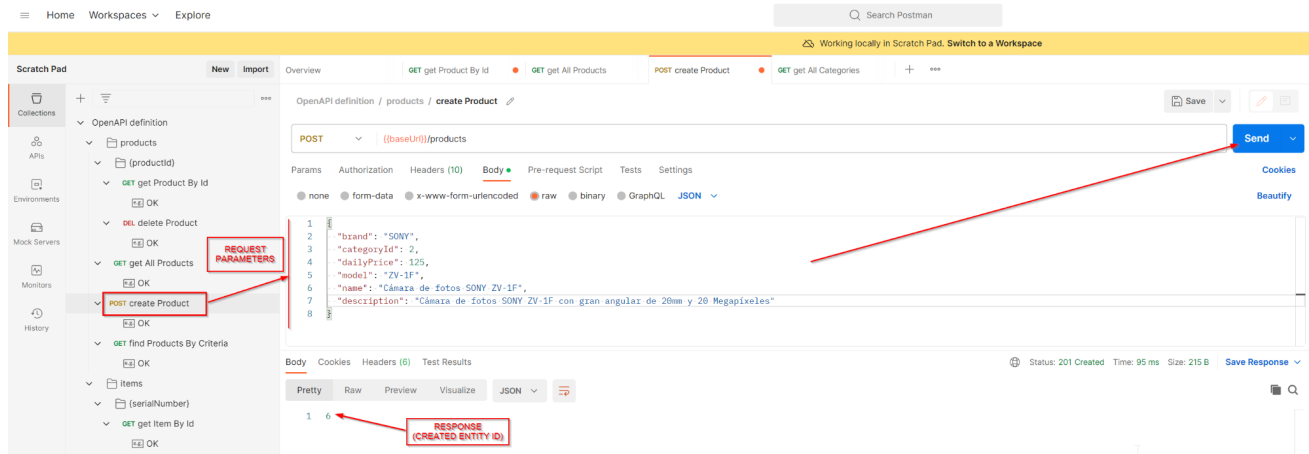
Este archivo JSON se puede descargar o importar directamente a una herramienta externa como Postman, desde dónde será mucho más sencillo realizar todas las pruebas necesarias para verificar el funcionamiento correcto de los diferentes servicios.





Desde esta vista podremos realizar cualquier llamada a las APIs desde un entorno gráfico y consultar los valores de retorno.





Ejercicio 2 (2 puntos)

El lanzamiento exitoso del proyecto **Photo&Film4You** ha despertado un gran interés entre varias empresas para establecer colaboraciones. Una de las propuestas que el equipo directivo está considerando es la presentada por el buscador de equipos para rodajes, **ShootMe**.

ShootMe, una plataforma líder en su sector, opera como un buscador unificado para diversos servicios de alquiler de equipos fotográficos, de rodaje, iluminación, entre otros. Si se llega a un acuerdo, esto representaría una oportunidad para alcanzar a una audiencia más amplia, a cambio de una pequeña comisión por cada referencia generada. La dinámica de funcionamiento implicaría que las consultas realizadas dentro de **ShootMe** consulten el catálogo de **Photo&Film4You**, junto con otros proveedores, y muestren los resultados pertinentes. Si un usuario muestra interés por alguno de los modelos disponibles para alquiler en **Photo&Film4You** y hace clic, será redirigido a nuestra aplicación con un código de afiliado, desde donde se gestionará el alquiler de manera habitual.

Aunque la dirección considera favorablemente esta propuesta, el Director de Tecnología (CTO) tiene algunas reservas. Durante una reunión técnica inicial con el CTO de **ShootMe**, se ha observado que su aplicación está basada en servicios que requerirán realizar consultas al catálogo de productos de **Photo&Film4You**. Sin embargo, la arquitectura de microservicios de Photo&Film4You presenta una elasticidad inherente, donde el número de instancias de cada microservicio fluctúa y ninguno posee una dirección IP o nombre fijo, lo que dificulta proporcionar un único punto de acceso al que la aplicación **ShootMe** pueda apuntar.

Considerando este escenario, se pide elegir y justificar la aplicación de un patrón para resolver el acceso a los microservicios de nuestra plataforma en las condiciones acordadas, y cuáles son los pros y contras de esta decisión.

Entregable

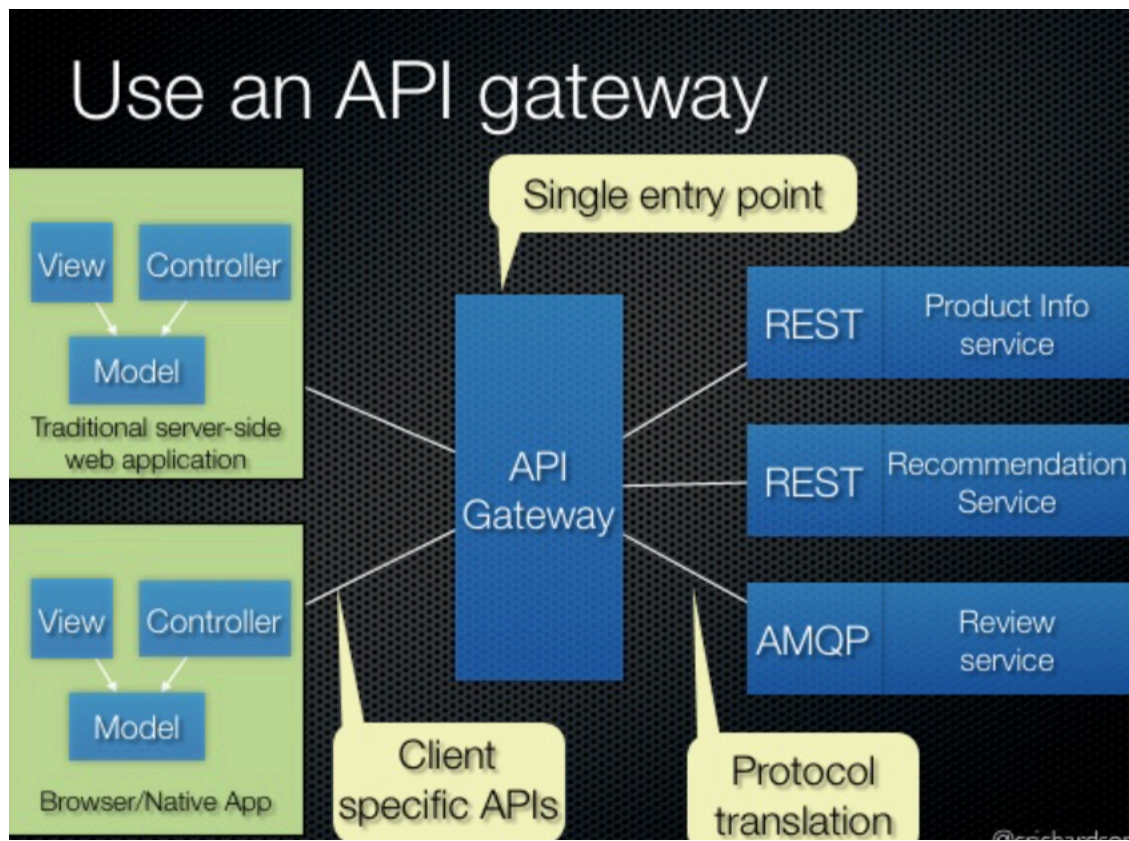
La respuesta a este ejercicio se entregará en formato texto PDF.

Solución

Nuestros colaboradores necesitan un único punto de entrada para realizar operaciones contra nuestros servicios.

Esta necesidad puede resolverse aplicando más de un patrón, dos patrones que nos pueden servir son *API Gateway* o *Service Registration/Service Discovery*. Cada opción tiene sus pros y sus contras y se aceptarán siempre que se justifiquen adecuadamente. En esta solución presentamos la justificación de la elección del patrón *API Gateway*.

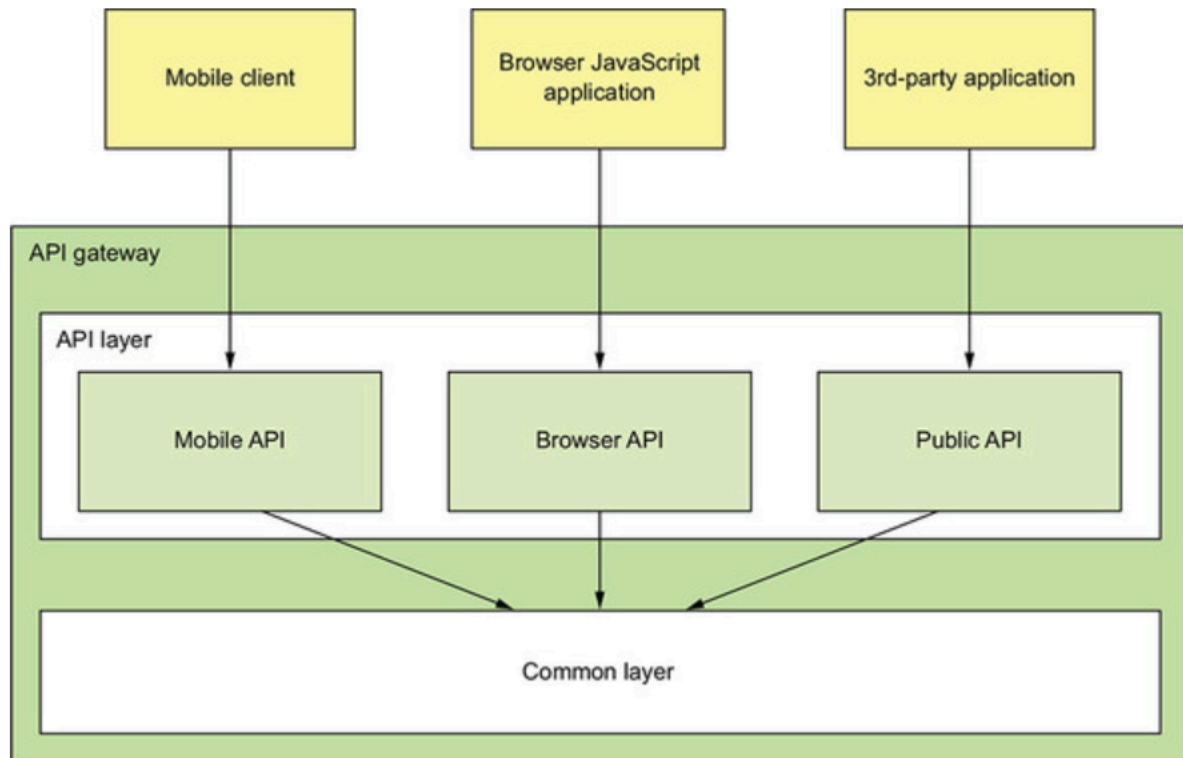
Una *API Gateway* es una pasarela que sirve de punto único de acceso a una aplicación desde el exterior. La aplicación de este patrón nos permite ofrecer una fachada al exterior que encapsula toda la arquitectura interna de forma similar a lo que se hace aplicando el patrón *Fachada*, bastante conocido dentro de la programación orientada a objetos.



Las funciones principales de este patrón son:

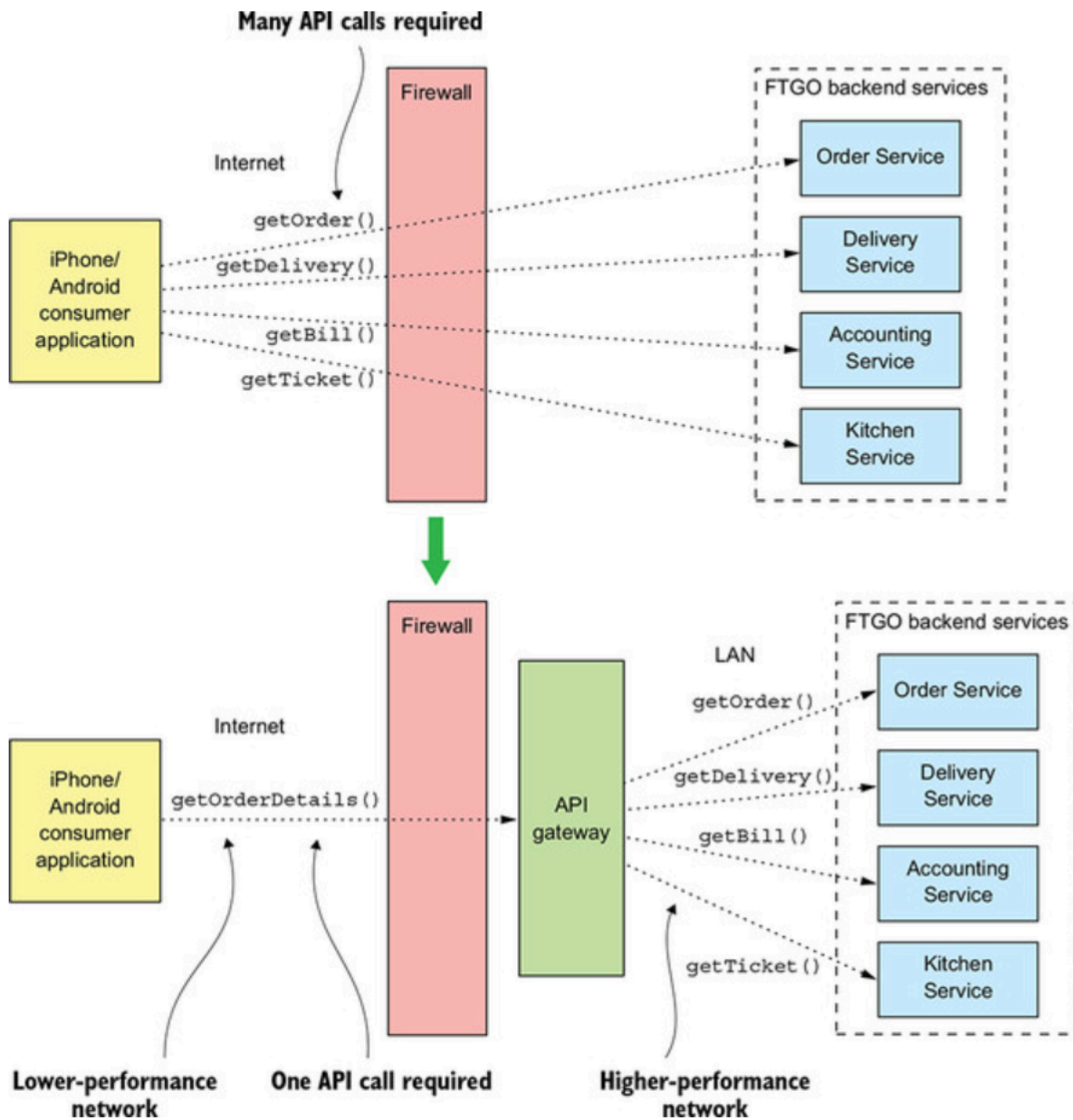
- El enrutamiento de las peticiones desde el punto de acceso único hacia los distintos servicios internos de que disponemos.
- La composición de las diversas *APIs* existentes en nuestro sistema.

Sin embargo, también existen otras ventajas. La aplicación de este patrón nos aporta muchísima flexibilidad y granularidad, ya que podemos optar por ofrecer una *API* diferente según la plataforma a la que queremos dar servicio, o diferentes *APIs* para distintos clientes concretos. Incluso podría ofrecerse una *API* de acceso público para aplicaciones de terceros, como hacen, por ejemplo, diversas aplicaciones de redes sociales (Twitter, Instagram, etc.).



También es posible, gracias a esta pasarela, gestionar las reglas de acceso, aplicar transformaciones a las peticiones, recoger métricas, etc.

En el siguiente diagrama se puede apreciar el uso de este patrón aplicado en el ejemplo FTGO del libro *Microservices Patterns*:



Como ocurre siempre en la arquitectura de software, en todas las decisiones hay un compromiso (*trade-off*), y como puntos negativos comparando con el uso de un *Service Discovery & Registration*, debemos decir que aunque *API Gateway* nos aporta mucha más funcionalidad, también implica unos costes mucho mayores y un aumento de la complejidad considerable, tanto a nivel lógico como a nivel de infraestructura.

Referencias:

- Microservices Patterns (Richardson). Chapter 8: "External API patterns".
- API Gateway: <https://microservices.io/patterns/apigateway.html>

Recursos

Recursos Básicos

- Libro "Microservices Patterns" (Chris Richardson): Chapter 6: "Developing business logic with event sourcing" (excepto sección 6.3).
- Libro "Microservices patterns" (Richardson): Chapter 7: "Implementing queries in a microservice architecture"
- Libro "Microservices patterns" (Richardson): Chapter 8: "External API patterns" (excepto sección 8.3).

Recursos Complementarios

- Repositorio de código fuente del ejemplo del libro Microservices Patterns. <https://github.com/microservices-patterns/ftgo-application>
- Ejemplo de programación de microservicios. <https://github.com/vmudigal/microservices-sample>

Criterios de evaluación

- En esta actividad **no está permitido el uso de herramientas de inteligencia artificial** y se tiene que resolver de forma **estrictamente individual**. En caso de detectar irregularidades se penalizará la actividad con una D como nota. Esto incluye la reutilización de soluciones de esta práctica de semestres anteriores. En el plan docente y en el [web sobre integridad académica y plagio de la UOC](#) encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.
- El peso de cada ejercicio está indicado en el enunciado.
- Los criterios de evaluación de cada ejercicio están indicados en el anexo.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Debe entregarse un único archivo ZIP con las respuestas a todos los ejercicios. El nombre del archivo debe ser: *ApellidosNombre_ISCSD_PRAC2.zip* con la siguiente estructura:

- ZIP con la entrega del ejercicio 1 parte 1, que debe contener:
 - ZIP con el código fuente
 - un archivo JAR ejecutable por cada microservicio implementado
 - un archivo readme.txt con instrucciones adicionales para arrancar la infraestructura y/o microservicios (especialmente en caso de cambios en la configuración o en el docker-compose)
- Colección Postman del ejercicio 1 parte 2.
- PDF de respuesta del ejercicio 2.

Este documento se entregará en el espacio “Entrega de la PRAC2” del aula antes de las **23:59 horas del día 24 de mayo de 2024**. No se aceptarán entregas fuera de plazo.

Anexo

Criterios de evaluación

	Excelente (10)	Notable (7)	Suficiente (5)	Insuficiente (2,5)	Sin respuesta / Todo mal (0)
Ej. 1 1a parte	• Número de operaciones no presentes en el esqueleto implementadas correctamente desde el punto de vista funcional (con 10 como máxima puntuación) • Uso correcto de los verbos HTTP • Control de errores (IDs que no existen, errores de tipo, etc.), validaciones y retorno del estado HTTP adecuado al resultado de la llamada • Colaboración completa entre servicios para llamadas REST • Utilizar todos los parámetros query/path según las	• Número de operaciones no presentes en el esqueleto implementadas correctamente desde el punto de vista funcional (mínimo 8) • Uso generalmente correcto de los verbos HTTP • Control de errores (IDs que no existen, errores de tipo, etc.), validaciones y retorno del estado HTTP generalmente adecuado al resultado de la llamada • Algunos problemas con la colaboración entre servicios para llamadas REST • Utilizar la mayoría de parámetros query/path	• Número de operaciones no presentes en el esqueleto implementadas correctamente desde el punto de vista funcional (mínimo 5) • Uso generalmente incorrecto de los verbos HTTP • Control de errores (IDs que no existen, errores de tipo, etc.), validaciones y retorno del estado HTTP generalmente inadecuado al resultado de la llamada • Muchos problemas con la colaboración entre servicios para llamadas REST • Uso de parámetros query/path, pero no siguen las	• Número de operaciones no presentes en el esqueleto implementadas correctamente desde el punto de vista funcional (menos de 5) • Uso incorrecto de los verbos HTTP • Control de errores (IDs que no existen, errores de tipo, etc.), validaciones y retorno del estado HTTP no adecuado al resultado de la llamada • No hay colaboración entre servicios para llamadas REST • No uso de parámetros query/path y/o no siguen las	Sin respuesta, no hay ninguna justificación o todo incorrecto (0)

	convenciones estándar • Utilizar DTOs para agrupar inputs/outputs • Justificación/explicación correcta en las partes donde sea necesario. • Calidad máxima de la implementación (legibilidad, formato, sin warnings, etc.) (6)	según las convenciones estándar • Utilizar DTOs para agrupar inputs/outputs • Justificación/explicación incorrecta o en lugares donde no es necesaria. • Calidad general aceptable de la implementación (legibilidad, formato, sin warnings, etc.) (4)	convenciones estándar • No utilización de DTOs para agrupar inputs/outputs • Ausencia de justificación/explicación. • Calidad general pobre de la implementación (legibilidad, formato, sin warnings, etc.) (3)	convenciones estándar • No utilización de DTOs • Calidad inaceptable de la implementación (legibilidad, formato, sin warnings, etc.) (1,5)	
Ej. 1 2a parte	Evidencias para todas las operaciones correctas, con información (entrada y salida) concreta para cada una de ellas (2)	Evidencias para aproximadamente 3/4 de las operaciones correctas, con información (entrada y salida) concreta para cada una de ellas. (1,5)	Evidencias para aproximadamente la mitad de las operaciones correctas, con información (entrada y salida) concreta para cada una de ellas. (1)	Evidencias para aproximadamente menos de la mitad de las operaciones correctas, con información (entrada y salida) concreta para cada una de ellas. (0,5)	Sin respuesta, no hay ninguna operación adicional respecto a las que ya tenía el esqueleto o no se puede importar el archivo de definición (0)
Ej. 2	Elección correcta del patrón y justificación completa y correcta que hace que esta elección sea la más adecuada (2)	Elección correcta del patrón, pero justificación incorrecta de que esta elección sea la más adecuada (1,5)	Elección incorrecta del patrón, pero justificación aceptable que hace que esta elección sea la más adecuada (1)	Elección incorrecta del patrón y sin justificación o incorrecta que hace que esta elección sea la más adecuada (0,5)	Sin respuesta, no hay ninguna elección ni justificación (0)